

Basic (your first real API call) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `api_first_call_practice.py` : every later exercise in this document assumes you've already seen, once, that a bad URL and a bad response are two completely different kinds of failure — one never reaches the network, the other comes back from a real server. This script is where you see that split for the first time, with nothing else going on. **If not:** the first time you'd meet that distinction would be buried inside a retry loop or an error-classification function, where it's much harder to isolate what's actually different about the two failure types.

Basic Version

Approach 1 — the direct way

```
# api_first_call_practice.py
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("http://broken")
except Exception as e:
    print("Caught it:", type(e).__name__)
```

Expected output:

```
200
{'current_user_url': 'https://api.github.com/user', ...}
Caught it: MissingSchema
```

This works. It catches `Exception` broadly rather than the specific `requests` exception class, which is fine for a first pass, but it would also swallow completely unrelated bugs in the same block if you added more code later.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the specific exception, wrapped in `main()`

```
# api_first_call_practice.py
import requests

def main() -> None:
    # why: proves a real call/response round trip works before anything
    # else in this document builds on top of it.
    response = requests.get("https://api.github.com")
    # how: the status code alone tells you the transport succeeded
    print(f"status: {response.status_code}")
    # how: .json() parses the response body – a separate step from the status
    # check
    print(f"body: {response.json()}")

    try:
        # when: a malformed URL fails before any network call even happens –
        # this is a client-side mistake, not a server response.
        requests.get("http://broken")
    except requests.exceptions.RequestException as e:
        # how: catches only requests' own exception family, not every possible
        # Exception – a bug elsewhere in a bigger try block still surfaces
        # normally.
        print(f"Caught a client-side request error: {type(e).__name__} - {e}")

if __name__ == "__main__":
    main()
```

Expected output:

```
status: 200
body: {'current_user_url': 'https://api.github.com/user', ...}
Caught a client-side request error: MissingSchema - Invalid URL 'http://broken': N
```

Difference from Basic: catching `requests.exceptions.RequestException` specifically (instead of bare `Exception`) means this block only swallows problems that are actually about making the request — a bug anywhere else in a bigger `try` block would still surface normally, instead of being silently caught here too. Wrapping the script in `main()` also matches the pattern from Doc01 — small habit, but it's the one you'll want once this script grows past a few lines.

Which one should you actually write? Intermediate's plain `try/except RequestException` is enough for exploring an API. This document's Build Task takes status-code classification (which failures to retry, which to fail on immediately) much further — that's the whole point of `http_client.py`, covered in its own exercises below, not something to over-build here.